

AN2DL - Second Challenge Report

overfittinghappens

Simone Sartori, Lorenzo Roberto Serraglio, Francesca Viggiani

simonesartori, loreserraglio, francescaviggiani

303656, 214128, 299452

June 21, 2026

1 Introduction

This project focuses on *images classification* using deep learning techniques. Our approach consisted of:

- Analyzing and cleaning the dataset;
- Developing a custom **CNN**;
- Gradually improving the CNN by applying Transfer Learning

2 Problem Analysis

The provided dataset consists of **1272 RGB images** of different sizes. Half of the data are microscopic pictures of diseased human tissue samples, labeled with **4 molecular subtype**: Luminal A, Luminal B, HER2-positive, and Triple Negative. Each picture has a corresponding **binary mask** that identifies the regions most likely to contain the diseased tissue (Figure 1).

We started our work by exploring the dataset and we spotted 2 main challenges:

- Even if the doctogres who collected the samples were really careful, we found some contaminated data. Data contamination could lead to misleading test performance so we developed some functions to detect and remove

all the pictures with ogre's material. We removed 110 images corrupted by artifacts.

- We observed some class imbalance that could harm performance on rare but important cases. We tried to address this problem later in the project.



Figure 1: example of an image, its mask, and the two overlapping

3 Method

3.1 Custom CNN

After exploring and cleaning the data, we split it into a training set (80%) and a validation set (20%). We started by designing a custom CNN that was trained from scratch for the classification task. The CNN consists in 4 blocks with an increasing number of filters. Each block is composed by:

- A **convolutional layer** to extract spatial features,

- An activation layer with **Relu** to introduce non-linearity,
- A **max pooling** layer to reduce spatial resolution.

The feature maps are then flattened and fed to a **dropout layer** for regularization and a **fully connected layer** that produces the final class logits. We tested different normalization approaches after the convolutional layers and **group normalization** turned out to be the most effective one. To improve generalization we added a simple **data augmentation** function that performs random transformations on both the image and its corresponding mask. The transformations include horizontal and vertical flips, rotations and a mild brightness and contrast adjustment (only on the images). We also employed a **patching** strategy. Patching allows images to be decomposed into smaller, fixed-size regions, helping the model focus on local visual patterns. We found this useful for our dataset, where relevant features are localized and a large portion of each image contains only background. Our method extracts 128×128 patches from tissue-containing regions and assigns higher importance to patches overlapping with the annotated regions of the mask. This approach encourages learning from the most discriminative areas while still preserving global context. At inference time, we employed **test time augmentation** to improve prediction robustness by averaging model outputs over multiple augmented versions of the same input. Our model tests different transformations and selects the one with the best performance on the validation set. Predictions from all augmented versions are then averaged, and patch-level outputs are combined using a max-confidence strategy.

3.2 Transfer Learning

We then decided to explore a transfer learning strategy (Figure ??) to exploit pretrained feature representations. Using models trained on large image datasets can provide a useful starting point, especially when the available dataset is relatively small. We tested two widely used architectures, **ResNet-34** and **DenseNet-121**.

3.3 ResNet-34

For ResNet-34, we explored several training configurations to improve robustness, starting from strategies to handle class imbalance. We tested different approaches:

- **Oversampling**: images from the Triple Negative class, which was the most underrepresented, were duplicated to reduce the imbalance across classes,
- **Label smoothing**: redistributing probability mass to non-target classes,
- **Weighted loss**: higher weights were assigned to minority classes, forcing the model to pay more attention to underrepresented samples during training.

We also tried a **MixUp augmentation**, combining images and labels from the training set through convex combinations. This led to better generalization, especially when paired with soft voting at inference time instead of relying on maximum confidence predictions. We then tested more complex strategies. A **branching architecture**, where the image and its corresponding mask were processed by parallel networks and later concatenated. This showed promising results but increased the complexity of the model. In the end we tried to implement a **GAN based image generation** using a pretrained Stable Diffusion inpainting model: new images were generated by keeping the masked region fixed while modifying the background with a “histopathology tissue” prompt. This approach improved validation performance but it was extremely computationally expensive, making this technique impractical to use.

3.4 DenseNet-121

We decided to explore the DenseNet-121 architecture for its parameter efficiency and dense connectivity, which promotes feature reuse and improves gradient propagation. We trained two DenseNet-121 models, differing mainly in their learning rate scheduling strategy. The first model employed a **Cosine Annealing Warm Restarts scheduler**, achieving higher peak validation performance but showing less stability during training. The second model used a **ReduceLROnPlateau scheduler**,

Table 1: Performance comparison

Model	Validation F1	Test F1
Custom CNN	0.3019	0.277
ResNet-34	0.3763	0.3220
DenseNet-121 (1)	0.3676	0.3254
DenseNet-121 (2)	0.3603	0.3335

resulting in slightly lower scores on validation but more consistent behavior, and a better performance on the test set ($F1 = 0.335$). Both models adopted a two-phase fine-tuning strategy. During the first phase, only the classifier was trained while the backbone remained frozen. In the second phase, part of the backbone was unfrozen, leading to approximately 2.4 million trainable parameters out of 7.2 million. Hyperparameters such as learning rates and weight decay were manually tuned.

4 Failed Experiments

During our experiments, we tested several techniques that did not lead to a performance improvement. Below we report the main approaches that were explored but eventually discarded during the building of our custom CNN

- **Complex data augmentation:** we initially implemented a more elaborate augmentation pipeline combining multiple geometric and color transformations. However, this approach did not improve performance, while a simpler and more lightweight augmentation strategy resulted in better generalization.
- **Mask-based patching:** we first attempted to extract patches exclusively from the masked regions. This approach didn’t allow us to get a sufficient number of patches, probably because of the very limited area of the masks.

During the experiments with **ResNet-34**, the

following strategies also failed to provide improvements:

- **Lion optimizer:** replacing Adam with Lion led to unstable training dynamics and did not improve performance.
- **Focal loss:** despite being designed to address class imbalance, focal loss did not yield better results in our setting.

5 Results

Through our systematic experimentation with various architectures, parameters and regularization strategies and regularization strategies, we were able to gradually improve our first model. The obtained scores are summarized in Table 1.

6 Conclusions

Our work shows that we were able to build a custom CNN that achieved competitive performance on the classification task. The use of transfer learning improved our validation scores and smoothed the validation loss/F1 curves, suggesting a reduction in overfitting. However, despite these improvements, the test predictions did not show a significant gain. This is likely because pre-trained representations sometimes require extensive adaptation to the target domain. Also, the moderate overall performance is probably due to the limited size of our dataset, that could have benefited from additional pre-processing steps.